

1. So in your `Employee` class, you marked `id`, `name`, `designation`, `salary` all as `private`. In your own words — what does `private` actually prevent? Like who specifically can't touch those fields once you do that?
2. You wrote a parameterized constructor for `Employee` — takes all four values at once. What would go wrong at the data level if you hadn't done that and instead had a no-arg constructor with four separate setter calls in `main`?
3. I see you wrote `getSalary()` and `getDesignation()` in `Employee`. What's the point of a getter method — why not just make the field `public` and skip the method entirely?
4. Your `main` uses a `Scanner` and reads employee data in a loop. You wrote `sc.nextLine()` right after `sc.nextInt()` for reading `num`. Can you walk me through why that extra line is there — what happens if you remove it?
5. You used `double` for `salary` and `int` for `id`. Simple question — what made you pick `double` for salary and not `int`? Was it a conscious decision or habit?
6. In your loop, you named the variables `a`, `b`, `c`, `d` for id, name, designation, salary. It works fine — but if your teammate opened this file tomorrow, what do you think their first reaction to `b` and `c` would be?
7. You printed the result like `"Average Salary : " + com.getAverageSalary(emp)`. That `+` between a string and a number — what is Java silently doing there for you behind that plus sign?
8. What does `emp.length` give you on an array? And in your loop `for (int i = 0; i < emp.length; i++)` — what would happen if you accidentally wrote `i <= emp.length` instead?
9. You passed `emp` into `Company`'s constructor and stored it as `employees`. Then you also passed `num` as `numEmployees`. But `emp.length` already knows the size — so why do you think the problem specifically asked you to store `numEmployees` separately?
10. In `getMaxSalary`, you'd need to loop through all employees to find the highest salary. What's the starting value you'd initialize your `max` variable to before the loop — and why does that starting value actually matter?
11. You called `com.getAverageSalary(emp)` and `com.getMaxSalary(emp)` — passing `emp` as a parameter even though `Company` already received and stored it in the constructor.

Looking at that now — does it feel a bit redundant? What would you need to change inside `Company` to make `com.getAverageSalary()` work without passing `emp` again?

12. `getEmployeesByDesignation` needs to return only employees matching a designation. But you're using a fixed-size array and don't know the match count ahead of time. How did you figure out how large to make the result array — and what would break if your guess was too small?

13. You used `.equalsIgnoreCase(designation)` for the search. Good call — but what happens to that line if any employee's stored designation is `null`? Where exactly does it break?

14. Your `Company` constructor takes `numEmployees` as a parameter. But what if the caller passes `numEmployees = 5` but the actual `emp` array only has 3 employees? Where in your methods would that mismatch silently cause a wrong answer — or crash?

15. Looking at your `Employee` class — it has getters but no setters. Was that deliberate? If a requirement came in saying "HR can update an employee's salary after joining," what would you add and where?

Tier 3 — What If / Conceptual

(~3-4 min each)

16. Suppose someone enters `salary = -80000` for an employee. Your constructor accepts it, no complaint. Where *should* that check live — constructor, setter, `main`, or somewhere else — and does the location actually matter?

17. What if the company grows and now has 500 employees? Your current `Employee[]` is fixed at creation time. If someone calls `addEmployee()` after the `Company` object is built — what two problems would you hit immediately with the current array design?

18. Suppose we need a `Manager` who is also an `Employee` but has an extra field `teamSize` and a `getBonus()` method. Would you modify `Employee` directly, copy-paste into a new class, or make `Employee` a base class? Walk me through your instinct and the reason behind it.

19. Right now `getEmployeesByDesignation` returns an array. Suppose the designation searched returns zero matches — what does your method return, and how does the calling code in `main` need to handle that carefully?

20. If you had to add a method `getTopEarner()` that returns the single `Employee` object with the highest salary — not just the salary number, but the whole `Employee` — what would the return type of that method be, and how is that different from `getMaxSalary()`?

Bonus — 5 Questions from PDF 1

B1. You wrote `com.getAverageSalary(emp)` — can I ask honestly, did it feel slightly off passing `emp` when `Company` already had it? What made you go with that approach?

B2. If we removed `numEmployees` from the constructor entirely and always derived it from `employees.length` — which line in `main` simplifies, and does anything else break?

B3. You print `"Average Salary: "` with the raw double value. If salary is `72500.0`, it prints `72500.0`. Did you consider showing it as `72,500.00`? What would you use to format it that way?

B4. Your `getEmployeesByDesignation` returns `null` or an array — the `main` has an `if(ans != null)` check. What prints if `ans` is an empty array (`length == 0`) instead of `null`? Would your current `if` catch that?

B5. If a colleague asked you — "Hey, why does `Employee` have getters but `Company` methods take `emp` as a parameter?" — how would you explain that design to them in two sentences?

PDF 2 — Bank Account Transfer System

Tier 1 — Foundational

(~1-2 min each)

1. In `BankAccount`, you have `acno`, `acname`, `balance` — all `private`. Same question I love asking — who is `private` actually keeping out, in practical terms?

2. You wrote getters for `acno`, `acname`, `balance` — but only a setter for `balance`. Why only `balance` gets a setter and not `acno` or `acname`? Was that intentional thinking or just what the problem needed?

3. `transferFunds` is marked `static` inside `BankUtils`. What does `static` mean for how you call it — do you need a `BankUtils` object to use it? How does the caller actually invoke it?

4. You used `throw new Exception("Insufficient Balance")` inside `transferFunds`. What does `throw` do in that line — what happens to the method's execution the moment that line runs?

5. Your `Transaction` class stores `code`, `amount`, `timestamp` — but with no `private`, no getters. What access level are those fields at by default, and what does that mean for any class trying to read `transaction.amount` directly?
 6. You used `new java.util.Date().toString()` for timestamp. What does that line actually produce — what does the string look like when printed? Day, time, timezone — what's in it?
 7. `BankUtils` has only one method and no fields. What kind of class is this conceptually — does it hold state, or is it just a home for utility logic? Why is that a reasonable design?
 8. In `main`, if `transferFunds` throws an exception, what needs to be around the method call to prevent the program from crashing? What Java structure handles that?
 9. Your `Transaction` constructor takes `code`, `amount`, `timestamp` in that order. If a caller accidentally passed amount and code swapped — `new Transaction(500.0, "TXN01", ...)` — what happens, and why doesn't Java catch that mistake automatically?
 10. `from.setBalance(from.getBalance() - amount)` — break that line down for me. What's `getBalance()` returning, what's being subtracted, and where does the result go?
-

Tier 2 — Applied & Specific to Your Code

(~2-3 min each)

11. You put `transferFunds` in a separate `BankUtils` class instead of inside `BankAccount`. What was your thinking there — why didn't it belong inside `BankAccount` in your mind?
12. `transferFunds` modifies both `from` and `to` accounts' balances. It does the debit first, then the credit. If an exception fires *between* those two lines — debit done, credit not done — what state is the system in? How would you describe that problem?
13. The problem asked you to throw `InsufficientBalanceException` — a custom exception. You used `Exception` instead. What's the practical difference for a caller trying to catch *only* the insufficient balance case versus other unexpected errors?
14. `Transaction`'s fields are package-private. `BankAccount`'s are `private` with getters. Looking at both side by side — which feels more consistent and why? What would you change to make them match?

15. Your `transferFunds` returns a `Transaction` object. But in `main`, does anyone actually use that return value — store it, print it, anything? What's the impact of ignoring a returned object in Java?

Tier 3 — What If / Conceptual

(~3-4 min each)

16. Suppose someone calls `transferFunds` with `amount = 0` or `amount = -500`. Your current check is only `from.getBalance() < amount`. Would either of those pass or fail that check? Where would you add a guard for it?

17. What if `BankAccount` needed to remember its last 5 transactions? What would you add to the `BankAccount` class to store them — one field, what type — and how would `transferFunds` need to change to record each transaction?

18. Suppose two people try to transfer from the same account at exactly the same moment — your current `setBalance` and `getBalance` are simple get/set. What kind of problem could that cause, and what concept exists to address it even if you haven't implemented it yet?

19. Right now `BankUtils` is a class with one static method. If the system grew to have 20 utility methods — `applyInterest`, `freezeAccount`, `auditTrail` — would you keep them all in one `BankUtils`, or split them? What would guide that decision?

20. Suppose we want `transferFunds` to *not* throw an exception but instead return a `boolean` — `true` for success, `false` for failure. How would the method signature change, what goes away, and what does the caller need to do differently?

Bonus — 5 Questions from PDF 2

B6. You used `throw new Exception(...)` — can I ask honestly, did you know how to create a custom exception class, or was `Exception` the quicker path? Either is fine — just curious.

B7. If I asked you to print `"Transfer of ₹[amount] successful"` only when the transfer actually completes — where in `transferFunds` is the exact right spot to print that, and why there?

B8. `Transaction` has a `timestamp` as a `String`. If you wanted to sort 10 transactions by time — earliest first — would string-based timestamp sorting work reliably? What could go wrong?

B9. Looking at your `BankAccount` — `setBalance` takes any double, including negative. What's one line you'd add inside `setBalance` to silently prevent the balance going below zero?

B10. If a junior asked you "why is `transferFunds` in `BankUtils` and not in `BankAccount`?" — what's your honest two-sentence answer to them?

PDF 3 — Beach Rating Finder

Tier 1 — Foundational

(~1-2 min each)

1. Your `Beach` class has `id`, `name`, `rating`, `cost` — all `private`. You exposed `getName()`, `getRating()`, `getCost()` but not `getId()`. Was leaving `getId()` out intentional, or did you just not need it for this problem?

2. `findLeastRatingWithName` is a `static` method sitting inside the `Answer` class. Why static here — could you have put this method inside `Beach` class instead? What would feel different about that?

3. You used `equalsIgnoreCase(e)` to match beach names. Simple one — what exactly does `equalsIgnoreCase` return, and how would the `if` condition behave if the stored name is "`Marina`" and the input is "`marina`"?

4. Your `rate` array starts as `new int[0]` — an empty array of size zero. What does `Arrays.copyOf(rate, rate.length + 1)` do to it the first time it's called? Walk me through what `rate` looks like before and after that first call.

5. You hardcoded `new Beach[4]` in `main`. The problem says there are 4 beaches — but where in the problem statement does it *guarantee* that number? What's the risk of that assumption?

6. `bc[i].getCost() > f` — that's a strict greater-than. If cost is exactly equal to `f`, does this beach get included or excluded? Is that what the problem asked for?

7. `Arrays.sort(rate)` — what does this do to the array, and after sorting, where is the minimum value sitting — index 0 or the last index?

8. Your method returns `0` when no match is found. And `main` checks `if(ans != 0)` to decide what to print. What's the problem if a beach *legitimately* has a rating of `0`?
9. You read beach data with `sc.nextInt()` then `sc.nextLine()` alternating. Why does `sc.nextLine()` keep appearing after `sc.nextInt()` — what would the `String b = sc.nextLine()` receive if you removed the `sc.nextLine()` before it?
10. Your `Beach` constructor takes `id`, `name`, `rating`, `cost` in that order. If the input is read in a different order — say `cost` before `rating` — what goes wrong, and where does the mismatch happen?
-

Tier 2 — Applied & Specific to Your Code

(~2-3 min each)

11. Inside your loop in `findLeastRatingWithName`, you call `Arrays.sort(rate)` every single time a new match is added. So if 6 beaches match, you sort 6 times on a growing array. What was your original intention with putting the sort inside the loop — and can you think of a way to sort only once, after the loop ends?
12. You start `rate` as `int[0]` and grow it with `Arrays.copyOf` on every match. Each `copyOf` creates a brand new array and copies everything over. If there were 100 matching beaches, how many array copy operations would happen — and does that concern you at all?
13. `findLeastRatingWithName` is placed as a static method in the `Answer` class rather than as a method in `Beach`. What's the argument for moving it into `Beach`? And what's the argument for keeping it where it is?
14. Your method returns `rate[0]` after sorting to get the minimum. But what if the requirement was to return the *Beach object* with the minimum rating, not just the integer rating — how would the return type and internal logic both need to change?
15. The `if(ans != 0)` check in `main` is your way of detecting "no results." But you also return `0` from the method when `rate.length == 0`. What's a cleaner signal for "nothing was found" that avoids the collision with a valid rating of `0`?
-

Tier 3 — What If / Conceptual

(~3-4 min each)

16. Suppose the requirement changes from "minimum rating" to "average rating of all matching beaches." Which exact lines inside `findLeastRatingWithName` change — the array growth logic, the sort, the return — and which stay exactly the same?

17. What if we need to find beaches where *both* name matches *and* cost is below a threshold instead of above? Walk me through exactly which part of the condition changes and whether anything else in the method is affected.

18. Right now `Beach` has no validation — `rating` can be `-5`, cost can be `0`, name can be an empty string. If you were asked to add a "valid beach" check before adding it to the array in `main`, where would that check go and what would it look like?

19. Suppose the array of beaches is replaced with an `ArrayList<Beach>` — what line in `main` changes, what changes inside `findLeastRatingWithName`, and what actually becomes easier to handle as a result?

20. Your method returns the minimum rating as an `int`. Suppose the business says "we actually want the *name* of the beach with the minimum rating, not the rating number." What needs to change in the return type, the internal tracking logic, and how `main` prints the result?

Bonus — 5 Questions from PDF 3

B11. You hardcoded `new Beach[4]` — did you know the input would always be 4, or did you just go with what the sample showed? If the count came from user input like in the Company problem, how would you read it?

B12. You put `Arrays.sort(rate)` inside the loop — can I understand your thinking? Were you sorting to find the minimum later, or did you have another reason in mind when you wrote it?

B13. If we change "find minimum rating" to "find all ratings above 3 for matching beaches" — which part of `findLeastRatingWithName` stays exactly the same, and which part needs a complete rewrite?

B14. Your `Beach` class is missing `getId()`. If the output needed to print "`Beach ID: 3, Rating: 4`" — what's literally the one thing you'd add to `Beach`, and where?

B15. Looking at your `findLeastRatingWithName` now — if you rewrote just the minimum-finding part without any array or sort, just a single `int min` variable tracked through the loop — how would that look? Can you sketch it out verbally?

